



兰州工业学院

实 验 报 告

(2024—2025学年第1学期)

课程名称 嵌入式系统应用

专业班级 专升本电信 23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验四 多线程应用程序设计		
实验时间	2024. 10. 23	实验地点	公共教学楼F312
一、实验内容 1. 了解Linux下多线程程序设计的基本原理 2. 学习pthread库函数的使用 3. 学习多线程间通讯的方法 4. 熟悉几个重要的 PTHREAD 库函数的使用，掌握互斥锁和条件变量在多线程间通讯的使用方法 5. 在ARM设备端使用NFS方式连接宿主机端试验目录，运行实验测试 二、实验器材（设备、元器件） 1. 个人计算机一台 2. Ubuntu Linux操作系统 3. IMX6嵌入式教学科研平台 三、实验步骤 1. 描述本次多线程程序设计的功能和思路 功能 资源共享：生产者线程负责生成数据并将其放入缓冲区，而消费者线程从缓冲区中取出数据进行处理。缓冲区是生产者和消费者共享的资源。 同步机制：为了保证数据的一致性和缓冲区的正确使用，需要同步机制来控制对共享资源的访问。这通常涉及到互斥锁（mutexes）和条件变量（condition variables）。 数据完整性：确保生产者不会在缓冲区满时写入数据，消费者也不会会在缓冲区空时读取数据。 结束信号：当生产者完成所有数据的生产后，需要有一种机制来通知消费者没有更多的数据将被生产，这通常通过一个特殊的结束标记来实现。 思路 初始化资源：创建一个共享缓冲区，并初始化读写位置指针。 生产者线程： 生产数据，直到达到预定的生产量。			

在每次生产数据后，检查缓冲区是否已满，如果满了则等待。

将数据放入缓冲区，并更新写位置指针。

通知消费者线程有新数据可用。

消费者线程：

检查缓冲区是否有数据可读，如果为空则等待。

从缓冲区取出数据，并更新读位置指针。

处理数据。

同步和互斥：

使用互斥锁来保护对缓冲区的访问，确保在任何时候只有一个线程可以修改缓冲区。

使用条件变量来同步生产者和消费者线程，当缓冲区满或空时，相应的线程会等待，直到条件改变。

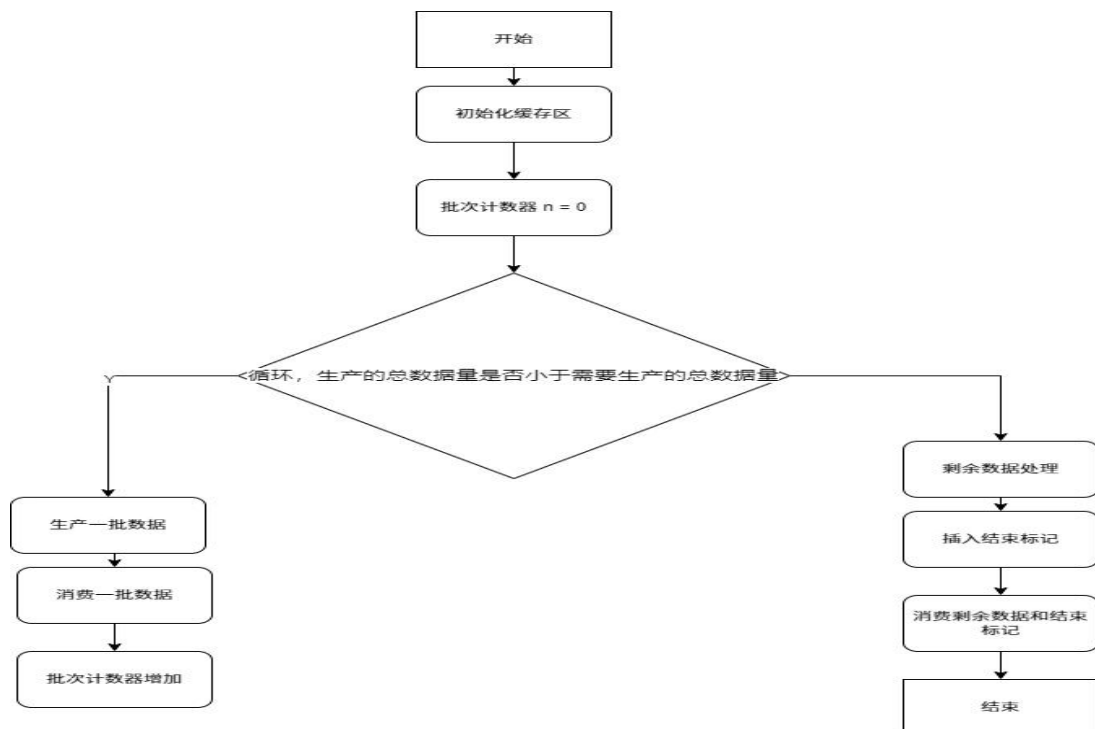
结束生产：

当生产者线程完成所有数据的生产后，将结束标记放入缓冲区。

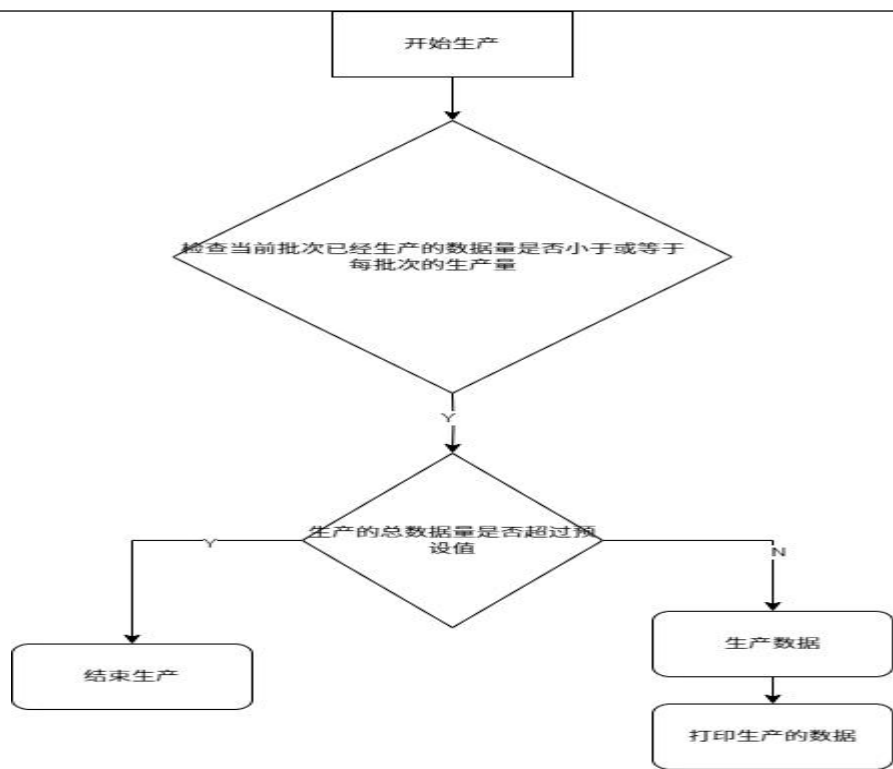
消费者线程在读取到结束标记时停止消费。

资源清理：在所有数据处理完成后，清理共享资源和同步机制。

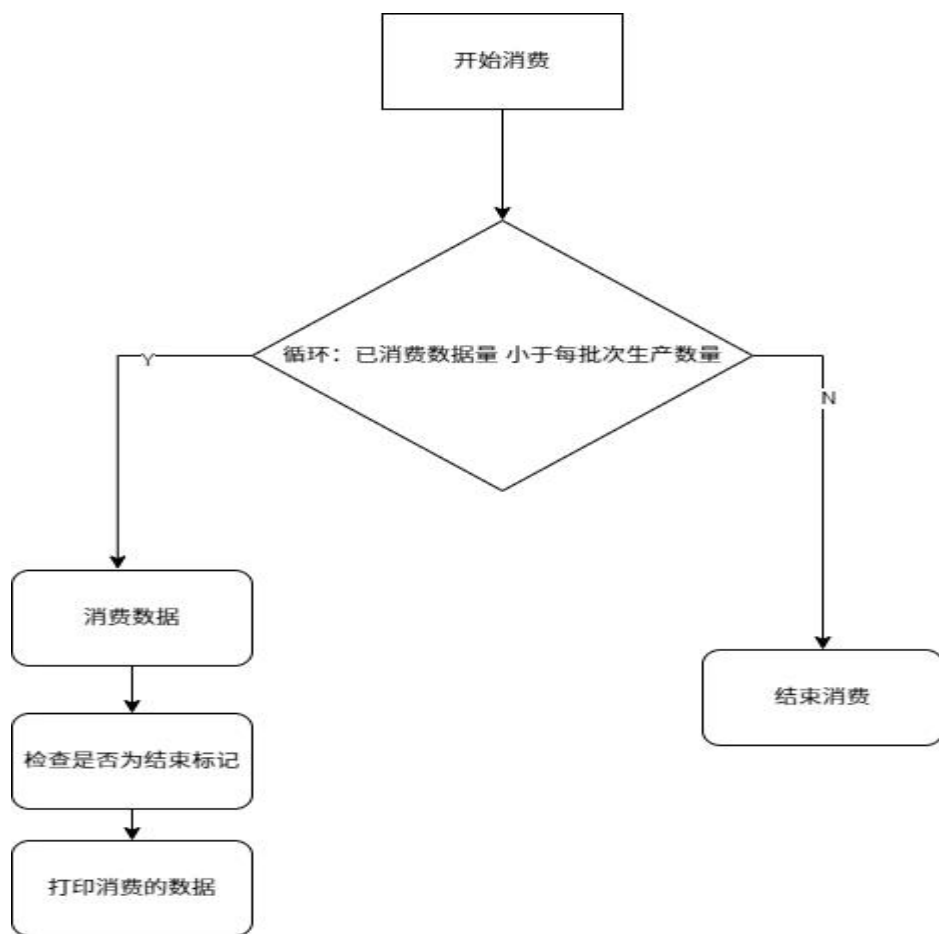
2.绘制多线程程序设计的流程图



图一 主程序



图二 生产者



图三 消费者

3. 在Ubuntu开发程序实现代码和编译



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 #define BUFFER_SIZE 16 // 定义缓冲区大小为16
6 #define BATCH_SIZE 16 // 定义每批次生产或消费的数据量为16
7 #define TOTAL_ELEMENTS 256 // 定义总共需要生产的数据量为256
8 #define END_MARKER -1 // 定义结束标记为-1, 用于告知消费者生产结束
9
10 struct prodcons {
11     int buffer[BUFFER_SIZE]; // 缓冲区数组, 用于存储生产者生产的数据
12     int readpos, writepos; // 读写位置指针, 分别表示消费者和生产者的位置
13 };
14
15 // 初始化缓冲区结构体
16 void init(struct prodcons * b) {
17     b->readpos = 0; // 初始化 读位置为0
18     b->writepos = 0; // 初始化写位置为0
19 }
20
21 // 生产者放入数据到缓冲区
22 void put(struct prodcons * b, int data) {
23     b->buffer[b->writepos] = data; // 将数据写入当前写位置
24 }
```

图 四 编写代码

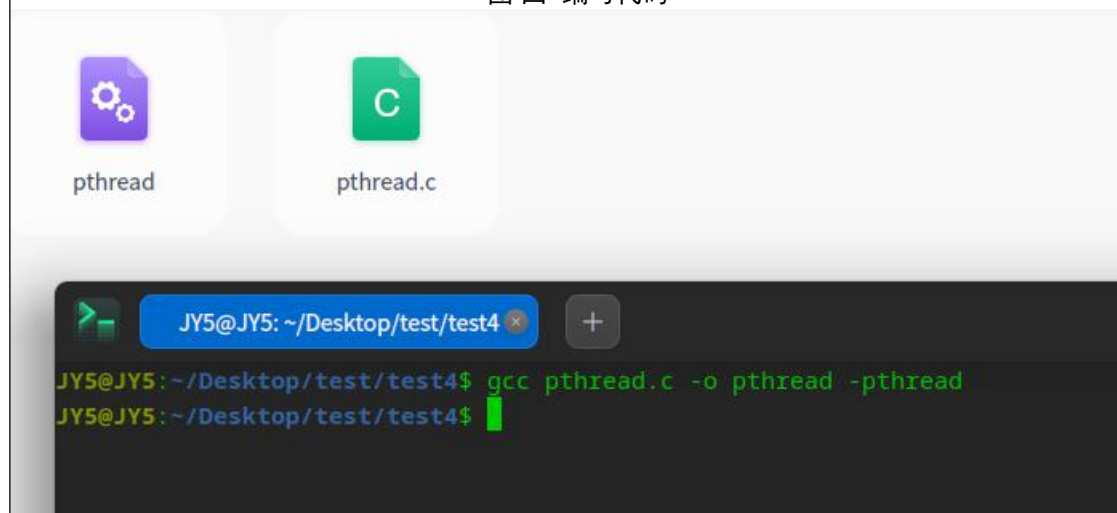


图 五 编译

4. 启动IMX6嵌入式实验平台使用NFS共享目录挂载运行测试

```
root@jy:/home/jy/桌面# mount -t nfs 192.168.16.2:/home/IMX6/ /mnt/
```

图 六 挂载

四、实验数据及结果分析

1. 交叉编译源代码

```
#include <stdio.h>

#include <stdlib.h>
```

```

#include <time.h>

#define BUFFER_SIZE 16 // 定义缓冲区大小为 16

#define BATCH_SIZE 16 // 定义每批次生产或消费的数据量为 16

#define TOTAL_ELEMENTS 256 // 定义总共需要生产的数据量为 256

#define END_MARKER -1 // 定义结束标记为-1，用于告知消费者生产结
束

struct prodcons {
    int buffer[BUFFER_SIZE]; // 缓冲区数组，用于存储生产者生产的数
据

    int readpos, writepos; // 读写位置指针，分别表示消费者和生产
者的位置
};

// 初始化缓冲区结构体

void init(struct prodcons * b) {
    b->readpos = 0; // 初始化 读位置为 0
    b->writepos = 0; // 初始化写位置为 0
}

// 生产者放入数据到缓冲区

void put(struct prodcons * b, int data) {
    b->buffer[b->writepos] = data; // 将数据写入当前写位置
    b->writepos++; // 写位置指针后移
    if (b->writepos >= BUFFER_SIZE) b->writepos = 0; // 如果写位置
超出缓冲区大小，回到起点
}

// 消费者从缓冲区取出数据

int get(struct prodcons * b) {
    int data = b->buffer[b->readpos]; // 从当前读位置取出数据
    b->readpos++; // 读位置指针后移
}

```

```

        if (b->readpos >= BUFFER_SIZE) b->readpos = 0; // 如果读位置超
        出缓冲区大小，回到起点

        return data; // 返回取出的数据
    }

    // 生产者函数，生产一批数据
    void producer(struct prodcons * b, int n) {
        for (int i = 1; i <= BATCH_SIZE; i++) {
            if (n * BATCH_SIZE + i > TOTAL_ELEMENTS) break; // 如果生
            产的总数据量超过预设值，则停止生产

            put(b, n * BATCH_SIZE + i); // 调用 put 函数生产数据
            printf(" 生产 -->%d\n", n * BATCH_SIZE + i); // 打印生产的
            数据
        }
    }

    // 消费者函数，消费一批数据
    void consumer(struct prodcons * b) {
        int consumed = 0; // 已消费的数据量
        while (consumed < BATCH_SIZE) {
            int d = get(b); // 调用 get 函数消费数据
            if (d == END_MARKER) break; // 如果消费到结束标记，则停止
            消费

            printf("          消费 -->%d\n", d); // 打印消费的数据
            consumed++; // 已消费的数据量增加
        }
    }

    int main(void) {
        struct prodcons buffer; // 定义缓冲区结构体
        init(&buffer); // 初始化缓冲区
    }

```

```

    int n = 0; // 批次计数器
    while (n * BATCH_SIZE < TOTAL_ELEMENTS) {
        producer(&buffer, n); // 生产一批数据
        consumer(&buffer); // 消费一批数据
        n++; // 批次计数器增加
    }

    // 处理不足一个批次的情况
    int remaining = TOTAL_ELEMENTS % BATCH_SIZE; // 计算剩余需要生
产的数据量
    if (remaining > 0) {
        for (int i = 0; i < remaining; i++) {
            put(&buffer, n * BATCH_SIZE + i); // 生产剩余的数据
            printf(" 生产 -->%d\n", n * BATCH_SIZE + i); // 打印生
产的数据
        }
    }

    put(&buffer, END_MARKER); // 在缓冲区末尾插入结束标记
    printf(" 生产 -->%d (结束标记)\n", END_MARKER); // 打印结束标
记

    consumer(&buffer); // 消费剩余的数据和结束标记
    return 0; // 程序结束
}

```

2.在ARM嵌入式Linux平台执行程序验证结果


```
JY5@JY5:~/Desktop/test/test4$ ./pthread
生产 -->1
生产 -->2
生产 -->3
生产 -->4
生产 -->5
生产 -->6
生产 -->7
生产 -->8
生产 -->9
生产 -->10
生产 -->11
生产 -->12
生产 -->13
生产 -->14
生产 -->15
生产 -->16
消费 -->1
消费 -->2
消费 -->3
消费 -->4
消费 -->5
消费 -->6
消费 -->7
消费 -->8
消费 -->9
消费 -->10
消费 -->11
消费 -->12
消费 -->13
消费 -->14
消费 -->15
消费 -->16
```

图 七 运行结果

五、总结

六、思考

什么是线程同步？Linux线程同步的实现方式有哪些？

线程同步是确保多个线程在访问共享资源时能够正确协调，以避免数据竞争和不一致状态的一种机制。在多线程编程中，线程同步非常重要，因为它可以防止多个线程同时修改同一数据，从而避免数据损坏。

Linux线程同步的实现方式：

互斥锁（Mutex）：

互斥锁是一种基本的同步机制，用于保护临界区。当一个线程进入临界区时，它会锁定互斥锁，防止其他线程进入临界区。当线程离开临界区时，它会释放互斥锁。

在 Linux 中，可以使用 `pthread_mutex_t` 来实现互斥锁。
读写锁（Read-Write Locks）：

读写锁允许多个读操作同时进行，但写操作是互斥的。这适用于读操作远多于写操作的场景。

Linux 提供了 `pthread_rwlock_t` 来实现读写锁。
条件变量（Condition Variables）：

条件变量用于线程间的协调，让一个或多个线程等待某个条件的发生。它通常与互斥锁一起使用，以避免竞态条件。

在 Linux 中，可以使用 `pthread_cond_t` 来实现条件变量。
信号量（Semaphores）：

信号量是一个计数器，用于控制对共享资源的访问。它可以用来控制同时访问某个特定资源的线程数量。

Linux 提供了 `sem_t` 来实现信号量。

实验成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与表现	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			刘建明 张翰茹

备注：考核项目及分值分布应依据教学大纲确定。